



Documentation de Validation

Projet Genie Logiciel - gl16

ENSIMAG 2025/2026

AIT MOHAMED Younes (aitmohay)

BEAURAIN Tom (beurato)

BEN ISMAIL Wajd (benismaw)

FURLANI DA SILVA SOARES Manuela (furlanim)

MARIA ENGLER PINTO Carlos (mariaenc)

Table des matières

1	Objectif de la validation	3
2	Descriptif des tests	3
2.1	Stratégie globale de test	3
2.2	Organisation des tests par étapes du compilateur	4
2.2.1	Étape A — Analyse lexicale et syntaxique	4
2.2.2	Étape B — Analyse contextuelle	4
2.2.3	Étape C — Génération de code	4
2.3	Organisation de la validation du codegen objet	5
2.4	Conformité, non-régression et maintenance	5
3	Types de tests par étape/passe	6
3.1	Étape A — Analyse lexicale	6
3.2	Étape A' — Analyse syntaxique	6
3.3	Étape B — Vérifications contextuelles	6
3.4	Étape C — Génération de code	7
4	Objectifs de test et manière dont ils sont atteints	7
4.1	Objectifs transverses	7
4.2	Atteinte des objectifs	7
5	Scripts de test	8
5.1	Lanceurs (wrappers)	8
5.2	Scripts de campagnes	8
6	Comment exécuter tous les tests	8
6.1	Pré-requis	8
6.2	Exécution globale	8
6.3	Exécution par famille	9
6.4	Infrastructure de validation et automatisation	9
6.4.1	Philosophie de l'automatisation	9
6.4.2	Les scripts lanciers (launchers)	9
6.4.3	Le script global : test-all-gencode.sh	9
6.4.4	L'oracle et la gestion des fichiers .res	10
6.4.5	Vérification de la non-régression	10
6.4.6	Intégration dans le cycle Maven	10
6.4.7	Gestion du risque et optimisation énergétique	10
7	Observations sur les échecs et limitations identifiés lors des tests	10
8	Gestion des risques et gestion des rendus	11
9	Couverture des tests (résultats de Jacoco)	12
9.1	Analyse globale de la couverture	12

10 Couverture des tests (résultats de Jacoco)	12
10.1 Analyse des résultats par paquetages	12
10.2 Impact de l'organisation des tests de codegen par axes	13
10.3 Interprétation de la couverture des branches	13
10.4 Méthode d'amélioration continue	13
11 Méthodes de validation autres que le test	14
11.1 Oracles manuels et fichiers de référence	14
11.2 Programmation défensive et assertions internes	14
11.3 Exécutions manuelles ciblées et débogage	14
11.4 Étude de l'idempotence de la décompilation (piste d'amélioration)	15
11.5 Revues de code et traçabilité des conventions	15
12 Conclusion	15

1 Objectif de la validation

Cette documentation décrit la stratégie de validation adoptée pour le compilateur `decac`. Elle est destinée à démontrer que l'implémentation est correcte et maintenable, tout en restant réaliste vis-à-vis des contraintes du projet.

Sans objet et avec objet. Notre compilateur supporte deux niveaux du langage Deca, qui ont fortement influencé l'organisation des tests. Le sous-ensemble *sans objet* constitue la base fonctionnelle : il couvre les types primitifs, les expressions, les affectations, les structures de contrôle et les entrées/sorties dans un bloc principal. L'extension *avec objet* ajoute les classes, l'héritage, les champs, les méthodes, l'allocation dynamique (`new`), la sélection `.`, les appels de méthodes et des constructions comme `this`, `null` et `instanceof`. Dans le dépôt, cette séparation se retrouve explicitement dans les répertoires et campagnes de tests (`sans_objet/`, `objet/`, `avec_objet/`). Le but n'est pas seulement de valider chaque niveau, mais aussi de garantir la *non-régression* : l'ajout de fonctionnalités objet ne doit pas dégrader le comportement correct du sans objet.

Interprétation des résultats et périmètre. La validation présentée ici suit les outils et jeux de tests fournis dans le cadre du projet, ainsi que des exécutions ciblées. Les résultats doivent être interprétés en tenant compte du périmètre effectivement implémenté : certaines fonctionnalités proposées peuvent ne pas faire partie de notre choix d'extension, et sont donc traitées comme des limitations connues. De même, certains comportements limites ne sont pas nécessairement couverts par les jeux de tests.

2 Descriptif des tests

2.1 Stratégie globale de test

La stratégie de validation adoptée repose sur un développement incrémental fortement guidé par les tests (*Test-Driven Development*, *TDD*). À chaque étape de l'implémentation du compilateur, des jeux de tests ont été conçus ou enrichis afin de valider les nouvelles fonctionnalités et de détecter au plus tôt les erreurs introduites.

L'objectif n'est pas uniquement de vérifier le bon fonctionnement du compilateur sur des cas simples, mais d'exhiber le plus grand nombre d'erreurs possibles, y compris sur des cas limites ou atypiques, afin de s'assurer qu'aucun défaut majeur ne subsiste dans la chaîne de compilation.

Tests boîte noire (fonctionnels). Les tests boîte noire sont élaborés exclusivement à partir des spécifications du langage Deca : grammaire concrète, sémantique informelle et grammaire attribuée. Ils ne tiennent pas compte de l'implémentation interne du compilateur. Leur objectif est de vérifier la conformité du comportement observé avec les règles du langage, aussi bien pour des programmes valides que pour des programmes invalides.

Tests boîte blanche (structurels). En complément, des tests boîte blanche visent à exercer des chemins spécifiques du code Java, notamment des parties critiques difficiles

à atteindre par des tests purement fonctionnels. Cela inclut par exemple la gestion de l'épuisement des registres (mécanismes de spill), les cas limites liés à la taille de la pile lors d'une récursion profonde, ou encore certains traitements d'erreur internes au compilateur.

2.2 Organisation des tests par étapes du compilateur

Les tests sont organisés de manière à valider chaque maillon de la chaîne de compilation, conformément à l'architecture en passes successives du compilateur `decac`. Cette organisation permet d'isoler plus facilement l'origine des erreurs et de limiter leur propagation d'une étape à l'autre.

2.2.1 Étape A — Analyse lexicale et syntaxique

Objectif. L'objectif principal est de vérifier la construction correcte de l'Arbre de Syntaxe Abstraite (AST) primitif à partir d'un fichier source `.deca`.

Tests. Les tests couvrent la lexicographie (gestion des chaînes de caractères, commentaires, mots-clés, identificateurs et littéraux) ainsi que la grammaire hors-contexte du langage. Ils incluent des fichiers valides et invalides afin de vérifier la détection d'erreurs syntaxiques localisées. Des tests spécifiques sont également dédiés à la directive `#include`, notamment pour vérifier le bon traitement des inclusions multiples et la détection d'inclusions circulaires.

2.2.2 Étape B — Analyse contextuelle

Objectif. Cette étape vise à valider les trois passes de vérification sémantique : construction de la hiérarchie des classes, vérification des signatures des champs et méthodes, et analyse des corps des méthodes et instructions.

Tests. Une utilisation intensive de tests classés `invalid/` permet de s'assurer que chaque règle de la grammaire attribuée est correctement appliquée. Ces tests vérifient notamment les erreurs de typage, les doubles déclarations, l'utilisation d'identifiants non définis ou encore les incompatibilités de signatures dans le cadre de l'héritage. En complément, des tests unitaires JUnit sont utilisés pour valider isolément certaines structures internes critiques, telles que la `SymbolTable` ou l'`EnvironmentExp`.

2.2.3 Étape C — Génération de code

Objectif. L'objectif est de garantir que le code assembleur produit pour la machine IMA est fidèle à la sémantique du langage Deca et qu'il s'exécute correctement sur l'interpréteur IMA.

Tests. Initialement, des tests de base ont été conçus pour valider la génération correcte du code pour des programmes simples utilisant des types primitifs, des affectations et des structures de contrôle telles que des boucles et des conditions. Par la suite, une attention particulière a été portée à la gestion de la pile et des registres, notamment à travers le mécanisme de spilling, qui permet de sauvegarder et restaurer des registres lorsque la

pile est saturée. Des tests ont aussi été effectués pour vérifier que le compilateur rejette correctement les programmes invalides, en s'assurant que les erreurs sont détectées et rapportées avec des messages d'erreur précis et localisés.

En cours de développement, de nouveaux tests ont été ajoutés pour couvrir des cas plus complexes et étendre la couverture des fonctionnalités. Chaque nouveau cas de test a permis de tester de nouvelles branches du code, validant non seulement les scénarios de réussite mais aussi les erreurs potentielles, ce qui a permis de vérifier la robustesse du compilateur face à différents types de programmes. Un aspect clé de cette phase était de garantir que chaque branche du code, en particulier les vérifications de conditions et les gestionnaires d'erreurs, était correctement exercée, afin de détecter des comportements inattendus et des erreurs de logique. Cela a permis d'assurer que le compilateur est capable de gérer une variété de cas, depuis les plus simples jusqu'aux plus complexes, avec une couverture de code améliorée à chaque ajout de test. Enfin, une série de tests de non-régression a été effectuée afin de garantir que l'ajout de fonctionnalités orientées objet ne générerait pas de régressions dans le comportement des programmes sans objet. Ces tests ont permis de valider que le compilateur génère un code valide, conforme aux spécifications du langage Deca, tout en maintenant la stabilité des fonctionnalités de base.

2.3 Organisation de la validation du codegen objet

Pour l'étape *C avec objet*, la validation de la génération de code a été structurée selon trois axes complémentaires, afin de couvrir de manière systématique les mécanismes propres à la programmation orientée objet.

Axe 1 — Table des méthodes (VTable) et héritage. Cet axe se concentre sur la construction des tables de méthodes (VTables) en zone globale (GB) et sur la résolution de la liaison dynamique. Les scénarios testés incluent l'héritage simple, la redéfinition de méthodes, le polymorphisme et les appels de méthodes via un adressage indirect dans la VTable.

Axe 2 — Champs et allocation mémoire. Les tests de cet axe portent sur la gestion des objets dans le tas (heap). Ils couvrent l'allocation via `new`, l'initialisation par défaut et explicite des champs, l'accès aux champs par sélection, les tests de nullité obligatoires, ainsi que le respect des règles de visibilité, notamment `protected`.

Axe 3 — Méthodes et blocs d'activation. Cet axe valide la gestion de la pile d'exécution (stack), le passage des paramètres, la gestion de `this` à l'offset `-2(LB)`, la récursion simple et mutuelle, ainsi que la préservation des registres non-scratch (R2-R15).

2.4 Conformité, non-régression et maintenance

Des tests de non-régression sont exécutés systématiquement afin de garantir que l'introduction des fonctionnalités orientées objet n'a pas dégradé le comportement du sous-ensemble sans objet (arithmétique, structures de contrôle, entrées/sorties).

Les tests sont classés de manière rigoureuse dans `src/test/deca/` selon une structure imposée : `valid/`, `invalid/`, et, pour la génération de code, `interactive/` (lecture

clavier) et `perf/` (mesure de performances). Chaque fichier `.deca` comporte un en-tête standardisé décrivant l'objectif du test et, le cas échéant, les résultats attendus, permettant une validation automatique à l'aide de fichiers de référence `.res`.

3 Types de tests par étape/passe

3.1 Étape A — Analyse lexicale

Type de tests. Il s'agit principalement de tests « boîte noire » : on fournit un fichier `.deca` au lanceur lexical, et on vérifie que :

- un fichier *valide* ne produit pas de message d'erreur standard (`fichier:ligne:...`);
- un fichier *invalide* produit un diagnostic lexical, localisé (ligne/colonne).

Ce niveau vise la robustesse des règles de tokenisation (chaînes, commentaires, littéraux, etc.) et la qualité des diagnostics.

3.2 Étape A' — Analyse syntaxique

Type de tests. L'analyse syntaxique est testée via un lanceur dédié exécutant le parseur et l'AST associé. La logique générale est :

- tests *valides* : l'analyse doit réussir sans erreur et sans exception;
- tests *invalides* : l'analyse doit échouer proprement, avec un message localisé.

Ces tests couvrent des points sensibles comme les priorités d'opérateurs, les parenthésages, les structures de blocs, et, côté objet, la syntaxe des classes, champs, méthodes, appels chaînés, cast, sélection (`.`), `new` et `instanceof`.

Cas limites identifiés lors des campagnes. Lors de l'exécution de `mvn compile test`, nous avons observé des cas classés *invalides* dans certains tests de référence (notamment sur des littéraux flottants extrêmes) qui ne sont pas systématiquement rejetés à ce stade dans notre implémentation. Ces écarts sont détaillés dans une section dédiée plus bas, car ils relèvent soit de choix de validation (pas d'analyse statique complète de certaines bornes), soit de limitations connues (crash sur certains littéraux trop grands).

3.3 Étape B — Vérifications contextuelles

Type de tests. Ces tests valident le typage, la portée des identifiants et les règles sémantiques. Ils sont plus fins que la syntaxe : un programme peut être bien formé syntaxiquement tout en étant rejeté contextuellement.

Les objectifs typiques sont :

- conformité du typage (affectations, opérations, conversions autorisées, conditions);
- cohérence des déclarations (double déclaration, variables non définies, etc.);
- côté objet : cohérence d'héritage, redéfinitions, compatibilité des signatures, visibilité (`protected/public`), usage de `this`, `return` dans les méthodes, et compatibilité `null`.

3.4 Étape C — Génération de code

Type de tests. La génération de code est validée via des tests système :

1. compilation `deca` → génération d'un fichier `.ass` ;
2. exécution du `.ass` sur l'outil IMA ;
3. vérification de l'absence de crash à l'exécution (et, lorsque pertinent, vérification de comportements attendus).

Dans notre dépôt, on distingue :

- des tests valides (le `.ass` doit être produit et exécutable) ;
- des tests invalides (la compilation doit échouer) ;
- des tests de performance (objectif : détecter des régressions grossières et des cas de stress).

Gestion des erreurs à l'exécution et contraintes de ressources. Les tests d'étape C exercent également la logique de détection d'erreurs à l'exécution (division par zéro, débordement flottant, débordement de pile). Ils reflètent aussi les contraintes inhérentes à la machine IMA : pile et tas de taille finie, nombre limité de registres, et impact de l'option `-r`. Ces limites ne sont pas des fautes de compilation, mais des contraintes d'exécution et de génération, et elles motivent une partie des tests de stress.

4 Objectifs de test et manière dont ils sont atteints

Notre stratégie de test vise à limiter les régressions et à isoler les erreurs au plus tôt. Elle repose sur le principe : plus un défaut est détecté tôt dans la chaîne, moins il coûte cher à corriger. Un point de vigilance spécifique au projet est la coexistence *sans objet* / *avec objet* : nous cherchons à garantir que les fonctionnalités avancées (objet) n'introduisent pas d'effets de bord sur le comportement de base (sans objet). Cela se traduit par des campagnes séparées, mais aussi par des exécutions complètes régulières (`execute-tests.sh`) afin de vérifier l'ensemble.

4.1 Objectifs transverses

- **Précision des diagnostics** : messages localisés et exploitables.
- **Robustesse** : pas d'exception Java non contrôlée sur une entrée invalide.
- **Non-régression** : conserver le bon comportement du sans objet lorsque l'objet évolue.
- **Couverture fonctionnelle** : cas simples + cas limites (priorités, héritage profond, chaînages).

4.2 Atteinte des objectifs

Les objectifs sont atteints par :

- une séparation claire `valid/invalid` dans chaque suite ;
- l'exécution automatisée via scripts et lanceurs ;
- l'ajout de tests lors de chaque fonctionnalité (en particulier sur l'objet) ;

- des revues croisées et une rotation des responsabilités, ce qui augmente la probabilité de détecter des incohérences entre modules.

5 Scripts de test

5.1 Lanceurs (wrappers)

Les exécutables `test_lex`, `test_synt` et `test_context` sont des wrappers shell situés dans :

```
src/test/script/launchers/
```

Ils construisent un classpath Maven (depuis `target/generated-sources/classpath.txt`) et lancent des programmes Java dédiés. Ils activent également les assertions Java (`-enableassertions`), ce qui permet de détecter des incohérences internes.

5.2 Scripts de campagnes

Les campagnes sont regroupées dans `src/test/script/`. Les scripts `test-all-lex.sh`, `test-all-synt.sh`, `test-all-context.sh` et `test-all-gencode.sh` exécutent les familles de tests (valid/invalid) et parcourent les sous-répertoires `provided`, `sans_objet` et `objet/avec_objet`. Un script racine `execute-tests.sh` orchestre l'exécution complète dans un ordre cohérent avec le pipeline du compilateur : on stabilise d'abord la syntaxe/lex, puis le contexte, puis le codegen.

Exécution Maven et lecture des échecs. En complément des scripts, l'exécution via Maven (`mvn compile test`) permet de lancer une partie des campagnes et de détecter rapidement des échecs récurrents. Les échecs observés dans ce mode ont été analysés et sont documentés ci-dessous afin de distinguer clairement ce qui relève d'un défaut du compilateur et ce qui relève d'une limitation ou d'un périmètre non visé.

6 Comment exécuter tous les tests

6.1 Pré-requis

Avant d'exécuter les tests, il faut que le projet soit compilé et que le classpath soit généré. Une séquence standard est :

```
mvn clean test-compile
```

Pour les tests de génération de code, l'outil `ima` doit être accessible (lancement du `.ass` généré).

6.2 Exécution globale

```
./execute-tests.sh
```

6.3 Exécution par famille

```
./src/test/script/test-all-synt.sh
./src/test/script/test-all-lex.sh
./src/test/script/test-all-context.sh
./src/test/script/test-all-gencode.sh
```

6.4 Infrastructure de validation et automatisisation

Cette section détaille l'infrastructure technique mise en place pour garantir la fiabilité du compilateur. L'automatisation est un pilier du projet, permettant des cycles de développement rapides et une détection immédiate des régressions.

6.4.1 Philosophie de l'automatisation

Le principe directeur est que chaque modification du code doit être validée par l'exécution de l'intégralité de la base de tests. Pour cela, nous utilisons des scripts shell qui servent d'intermédiaires entre le code source Deca et la machine **IMA**, permettant de transformer une vérification manuelle fastidieuse en un processus systématique.

6.4.2 Les scripts lanciers (launchers)

Pour valider les étapes intermédiaires (A et B), nous utilisons les scripts fournis dans `src/test/script/launchers` :

- **test_lex** : Isole l'analyseur lexical pour vérifier la transformation du flux de caractères en lexèmes.
- **test_synt** : Valide la construction de l'Arbre de Syntaxe Abstraite (AST) primitif.
- **test_context** : Vérifie la sémantique (typage, environnements) et produit l'AST décoré.

Ces scripts sont essentiels pour le débogage ciblé avant d'entrer dans la phase de génération de code.

6.4.3 Le script global : `test-all-gencode.sh`

Ce script constitue le cœur de notre processus de validation pour l'étape C. Il automatise le cycle complet "Compilation-Exécution-Vérification" pour les tests de notre base.

Structure et fonctionnement Le script parcourt récursivement les dossiers de tests organisés par axes :

1. **Phase de nettoyage** : suppression des anciens fichiers `.ass`.
2. **Compilation via decac** : appel du compilateur sur le fichier `.deca` et vérification de l'existence du fichier assembleur généré.
3. **Exécution via IMA** : l'interpréteur exécute le fichier `.ass`.
4. **Gestion des erreurs** : capture des crashes d'IMA et affichage avec codes couleurs pour lecture immédiate.

6.4.4 L'oracle et la gestion des fichiers .res

L'****Oracle**** décide si un test est réussi. La sortie standard d'un programme valide est stockée dans un fichier **.res**. Le script compare cette sortie avec le fichier de référence validé manuellement. Les écarts signalent un échec. Pour les dossiers **invalid/**, l'oracle vérifie que le compilateur lève bien une erreur et ne produit pas de code.

6.4.5 Vérification de la non-régression

Afin de garantir que l'ajout de l'orienté objet (Étape C) ne détruit pas les fonctionnalités de base :

- Le script **test-all-gencode.sh** relance systématiquement les tests **sans objet**.
- Cela assure la compatibilité des structures de contrôle (**while**, **if**) et de l'arithmétique simple avec la nouvelle gestion de la pile globale et des VTables.

6.4.6 Intégration dans le cycle Maven

Tous les scripts sont rattachés à la phase de test de Maven via le **pom.xml** :

- **mvn verify** : recompilation, instrumentation Jacoco et exécution de tous les scripts shell et tests unitaires.
- Codes de retour : **exit 0** si succès, **exit 1** sinon, bloquant le processus Maven en cas d'erreur.

6.4.7 Gestion du risque et optimisation énergétique

Les scripts ont été optimisés pour limiter le coût énergétique de la validation. Seuls les tests nécessaires sont exécutés, tout en maintenant un objectif de fiabilité maximale, respectant ainsi le compromis demandé par le cahier des charges.

7 Observations sur les échecs et limitations identifiés lors des tests

Cette section documente explicitement les échecs observés lors de l'exécution de **mvn compile test** et leur interprétation. L'objectif n'est pas de modifier les résultats, mais de clarifier ce qui relève d'un bug, d'une limitation connue ou d'un choix de périmètre.

Tests syntaxiques : float_underflow.deca et float_overflow.deca. Les fichiers **float_underflow.deca** et **float_overflow.deca** sont classés comme invalides dans les tests de syntaxe. Dans l'état actuel du compilateur, ces cas ne sont pas rejetés au niveau syntaxique ou contextuel de manière systématique, car la gestion fine des sous-dépassements/dépassements flottants n'est pas traitée comme une vérification statique complète à la compilation. Ces situations sont principalement destinées à être traitées dynamiquement, au moment de l'exécution (Partie C), via des mécanismes d'erreur à l'exécution.

Crash sur des littéraux flottants extrêmement grands. En plus du point précédent, nous avons identifié une limitation : le compilateur peut *crasher* sur certains littéraux flottants trop grands. L'erreur observée est cohérente avec l'intuition de dépassement de capacité, mais elle n'est pas rapportée proprement sous forme d'un diagnostic `fichier:ligne:colonne`. Ce comportement est donc à considérer comme une limitation connue (robustesse sur cas extrêmes) et non comme un rejet contrôlé.

Tests contextuels : répertoire `not_sorted`. Lors de l'exécution des tests contextuels, une erreur est observée car le répertoire `not_sorted` ne contient pas directement de fichiers `.deca` au niveau attendu : il contient uniquement des sous-répertoires. Le framework de tests s'attend à trouver des fichiers `.deca` à ce niveau, ce qui provoque une erreur lors de l'exécution globale. Les fichiers présents dans les sous-répertoires, notamment les tests de cast, passent lorsqu'ils sont exécutés dans leur contexte attendu.

Tests de génération de code : `exmathdoc.deca`. Le test `exmathdoc.deca` échoue avec un message de type « not yet implemented ». Cet échec est attendu : ce test repose sur l'extension `TRIGO`, qui n'a pas été implémentée dans ce projet. À la place, l'équipe a choisi de développer une extension alternative : une génération de code ciblant l'architecture ARM. Par conséquent, les fonctionnalités mathématiques avancées associées à `TRIGO` ne sont pas disponibles et l'échec de `exmathdoc.deca` reflète une différence de périmètre.

Limitations liées aux ressources IMA (`pile`, `tas`, `registres`). Certaines limitations sont inhérentes à l'exécution sur IMA et doivent être prises en compte lors de l'interprétation des tests système : l'exécution est limitée par la taille de la pile allouée par l'interpréteur IMA (10 000 mots par défaut), une récursion trop profonde provoque une erreur `pile_pleine`. De même, l'allocation d'un trop grand nombre d'objets via `new` peut saturer le tas et déclencher `tas_plein`. Enfin, lorsque l'utilisateur limite les registres via l'option `-r 4`, l'évaluation d'expressions très complexes peut devenir plus sensible à l'efficacité du mécanisme de spilling (sauvegarde/restauration sur pile), sans que cela constitue une erreur de correction du compilateur. Bien que la base de tests couvre les cas standards et les erreurs sémantiques répertoriées, des scénarios très extrêmes (structures de classes très imbriquées, expressions dépassant la capacité de stockage temporaire) pourraient révéler d'autres limitations non couvertes.

8 Gestion des risques et gestion des rendus

La gestion des risques a été intégrée dans la validation car un compilateur est un pipeline : un défaut peut se propager d'une étape à l'autre. Le risque principal est la non-régression, notamment entre les deux niveaux *sans objet* et *avec objet* : l'ajout de cas objet peut casser des tests historiques sans objet si les conventions internes ou les chemins de code communs ne sont pas correctement isolés.

Les mesures de mitigation ont été :

- développement incrémental et intégrations régulières ;
- campagnes de tests reproductibles (scripts) ;
- revues croisées et rotation des rôles ;

- branche principale maintenue compilable autant que possible.

Gestion des rendus et transparence sur le périmètre. Afin de maîtriser le risque lié aux extensions et aux attentes des tests fournis, nous avons explicité dans la documentation utilisateur les choix d’extension, et nous documentons ici les échecs directement liés à ces choix (cas `TRIGO` vs extension `ARM`). Cette transparence évite d’interpréter un échec comme un défaut de compilation, lorsqu’il reflète en réalité un périmètre non visé.

9 Couverture des tests (résultats de Jacoco)

L’évaluation de la qualité d’une base de tests ne peut se limiter au nombre de fichiers ‘.deca’ exécutés; elle nécessite une mesure objective de la sollicitation du code source Java par ces tests. Pour ce faire, nous avons utilisé l’outil Jacoco, intégré à notre cycle de construction via la commande ‘`mvn verify`’.

9.1 Analyse globale de la couverture

Notre stratégie de validation a priorisé les paquetages gérant la sémantique et la traduction en code machine, car ils contiennent la logique la plus complexe et les risques de régressions les plus élevés. Les résultats obtenus témoignent d’une couverture très élevée sur les composants essentiels du compilateur :

- **fr.ensimag.deca.context (86 %)** : Ce taux très élevé garantit que la quasi-totalité des règles de la grammaire attribuée (typage, vérification des environnements, passes 1, 2 et 3) a été testée de manière exhaustive.
- **fr.ensimag.deca.tree (82 %)** : Ce paquetage, qui définit la structure de l’Arbre de Syntaxe Abstraite (AST), est sollicité à chaque étape du processus. Une telle couverture assure la fiabilité des méthodes de décoration (‘`verifyExpr`’, ‘`verifyInst`’) et de génération de code (‘`codeGenExpr`’).
- **fr.ensimag.ima.pseudocode (79 %)** : Ce résultat valide notre capacité à produire une grande variété d’instructions pour la machine abstraite IMA, couvrant les modes d’adressage et les opérations arithmétiques ou de contrôle.

10 Couverture des tests (résultats de Jacoco)

L’évaluation de la fiabilité de notre compilateur s’appuie sur une mesure objective de la couverture du code Java par notre suite de tests Deca. Pour ce faire, nous utilisons l’outil **Jacoco**, qui analyse quelles instructions et quelles branches logiques ont été effectivement exécutées lors de la phase de test.

10.1 Analyse des résultats par paquetages

Les résultats globaux montrent une couverture solide de **78 % des instructions** et **65 % des branches** sur l’ensemble du projet. L’analyse détaillée par paquetage met en évidence la robustesse des étapes cruciales de la compilation (Étapes B et C) :

- `fr.ensimag.deca.context` (**86 % des instructions, 70 % des branches**) : Il s'agit du taux le plus élevé du projet. Cette couverture quasi exhaustive garantit que les trois passes de vérification sémantique (hiérarchie des classes, signatures des membres et corps des méthodes) sont validées pour une grande diversité de scénarios.
- `fr.ensimag.deca.tree` (**82 % des instructions, 75 % des branches**) : Ce paquetage, cœur de l'Arbre de Syntaxe Abstraite (AST), présente une excellente couverture. Cela confirme que les méthodes de décoration et de génération de code intégrées aux nœuds de l'arbre via le patron « interprète » sont massivement sollicitées.
- `fr.ensimag.ima.pseudocode` (**79 % des instructions, 75 % des branches**) : Ce résultat démontre que notre base de tests parvient à couvrir la majeure partie de la logique de manipulation des instructions de la machine abstraite **IMA**.

10.2 Impact de l'organisation des tests de codegen par axes

L'obtention de taux de couverture supérieurs à 80 % dans les paquetages `tree` et `context` est le résultat direct de notre stratégie de test segmentée en **trois axes fonctionnels** pour la partie objet :

1. **Axe 1 : Table des méthodes et héritage** : En testant systématiquement la construction de la `VTable`, le polymorphisme et la liaison dynamique, nous avons sollicité les branches complexes des classes `DeclClass` et `MethodCall`.
2. **Axe 2 : Champs et allocation** : Ces tests ont permis de couvrir les nœuds de l'AST relatifs à l'instruction `NEW`, à la sélection de champs et à la visibilité `protected`, validant ainsi la gestion du tas et les offsets mémoire.
3. **Axe 3 : Méthodes et blocs d'activation** : Cet axe a été déterminant pour tester la gestion de la pile, la récursion et les mécanismes de **saturation des registres (spill)** via des expressions complexes, forçant l'exécution des instructions `PUSH` et `POP` dans `AbstractBinaryExpr`.

Cette approche par axes a permis d'isoler chaque fonctionnalité sémantique du langage complet, assurant qu'aucun nœud de l'arbre abstrait ne reste inexploré lors de la génération de code.

10.3 Interprétation de la couverture des branches

Le taux de couverture des branches (**65 % au total**) est structurellement inférieur à celui des instructions. Cela s'explique par la nature du compilateur qui intègre une part importante de **programmation défensive**. De nombreuses branches conditionnelles, notamment les vérifications de préconditions via `Validate.notNull` ou les `assert`, ne sont empruntées qu'en cas d'erreur interne du compilateur ; elles restent donc « rouges » tant que le compilateur se comporte de manière stable.

10.4 Méthode d'amélioration continue

Jacoco a servi de guide pour le développement de notre base de tests. En identifiant les « trous » de couverture, nous avons pu :

1. Ajouter des tests ciblés pour les cas d’erreurs contextuelles rares.
2. Vérifier la **non-régression** entre le langage « sans objet » et le langage complet, garantissant que l’ajout des classes n’altérerait pas les bases du compilateur.

11 Méthodes de validation autres que le test

Outre l’exécution systématique de programmes Deca à l’aide de tests automatisés, nous avons mis en œuvre plusieurs méthodes complémentaires afin de garantir la qualité et la robustesse du compilateur. Ces approches couvrent à la fois la validation manuelle, l’inspection du code, l’utilisation d’assertions internes et l’exploration de propriétés plus formelles du compilateur.

En complément des tests automatisés, nous avons notamment utilisé :

- la relecture et l’inspection du code (revues de code) ;
- des assertions internes permettant de détecter la violation d’invariants pendant l’exécution ;
- des exécutions manuelles ciblées sur des cas complexes à des fins de débogage ;
- une traçabilité minimale des conventions internes, facilitant l’intégration et la maintenance.

11.1 Oracles manuels et fichiers de référence

Pour chaque test fonctionnel de la partie génération de code, nous avons adopté une approche de **validation par comparaison**. Chaque nouveau test a d’abord été exécuté et analysé manuellement afin de vérifier que la sortie produite par la machine abstraite **IMA** était conforme aux attentes sémantiques du langage Deca.

Une fois ce comportement validé, la sortie a été archivée dans un fichier de référence `.res`. Ces fichiers servent ensuite d’oracles de référence pour la détection de régressions. Le script `test-all-gencode.sh` compare automatiquement les sorties courantes avec ces fichiers à l’aide d’un `diff`, transformant ainsi une validation initialement manuelle en un mécanisme de non-régression entièrement automatisé.

11.2 Programmation défensive et assertions internes

Nous avons intégré la **programmation défensive** directement dans le code source Java du compilateur. L’usage systématique de vérifications de préconditions, telles que `Validate.notNull()` et `Validate.isTrue()`, permet de détecter très tôt des incohérences internes, avant qu’elles ne conduisent à des erreurs plus complexes à diagnostiquer.

Par ailleurs, l’utilisation d’assertions (`assert`) a permis de vérifier la cohérence structurale de l’Arbre de Syntaxe Abstraite (AST) tout au long des différentes passes de l’analyse contextuelle (étape B). Ces assertions jouent le rôle d’invariants internes et constituent un outil efficace de détection d’erreurs pendant l’exécution des tests.

11.3 Exécutions manuelles ciblées et débogage

En complément des tests automatisés, nous avons eu recours à des **exécutions manuelles ciblées** sur des programmes Deca complexes. Cette démarche a été particulière-

ment utile pour analyser finement le comportement du compilateur dans des situations délicates, notamment lors de la génération de code.

L'observation directe du code assembleur produit et de son exécution sur la machine IMA a permis de détecter et de corriger des erreurs subtiles liées, par exemple, à la gestion de la pile, à l'allocation mémoire ou à l'utilisation des registres, qui ne se manifestaient pas nécessairement par des échecs de tests automatisés.

11.4 Étude de l'idempotence de la décompilation (piste d'amélioration)

Une propriété cruciale spécifiée pour le projet est la **sobriété de la décompilation**, qui impose que la décompilation soit idempotente. Autrement dit, si $P2$ est obtenu par décompilation de $P1$, alors la décompilation de $P2$ doit produire un programme $P3$ tel que $P2 = P3$.

Bien que le compilateur ait été fonctionnel dans les délais du rendu principal, nous avons exploré cette piste de validation dans une phase ultérieure. La démarche consistait à générer un programme $P2$ à l'aide de `decac -p P1`, puis un programme $P3$ via `decac -p P2`, avant de comparer $P2$ et $P3$ à l'aide d'un `diff`.

Cette analyse a permis de mettre en évidence certaines failles subtiles dans la structure de l'AST et dans la gestion des parenthèses dites de courtoisie, qui n'avaient pas été révélées par les tests fonctionnels classiques. Cette méthode constitue ainsi un oracle automatique très fin, que nous considérons comme une piste essentielle pour atteindre un haut niveau de fiabilité.

11.5 Revues de code et traçabilité des conventions

Enfin, conformément aux principes du génie logiciel, nous avons pratiqué des **revues de code** croisées au sein de l'équipe. Elles ont été particulièrement efficaces pour valider la logique de la table des symboles et la gestion des environnements, avant l'entrée dans la phase de codage intensif de la génération de code.

Nous avons également maintenu une **traçabilité minimale des conventions internes** (conventions d'appel, organisation mémoire, utilisation des registres), ce qui s'est révélé utile lors des phases d'intégration, de refactorisation et de maintenance du compilateur.

Validation manuelle sur les cas d'échec. Les échecs observés via `mvn compile test` ont été reproduits manuellement afin de confirmer leur cause. Cela permet de séparer (i) les limitations connues (crash sur certains littéraux flottants très grands, périmètre TRIGO non implémenté) de (ii) des erreurs de configuration du framework (structure `not_sorted`) et de (iii) les contraintes d'exécution propres à IMA.

12 Conclusion

La validation de `decac` s'appuie sur une campagne structurée par étape (lex, syntaxe, contexte, codegen), sur des suites `valid/invalid` et sur des scripts reproductibles.

La séparation explicite entre *sans objet* et *avec objet* permet de suivre l'avancement de chaque niveau du langage, tout en garantissant la non-régression lors de l'introduction de fonctionnalités avancées (classes, héritage, vtables, allocations et appels dynamiques). La couverture JaCoCo complète cette approche en fournissant une mesure objective des zones réellement exercées et en guidant l'amélioration continue des tests.

Bilan des limites documentées. Les limites observées lors des tests automatisés ont été analysées et documentées : cas flottants extrêmes, organisation de certains tests de référence, et divergence volontaire sur les extensions (TRIGO vs ARM). Ces points clarifient le périmètre réellement couvert et facilitent l'évaluation du projet dans un cadre cohérent avec les choix d'implémentation.